

devteam: a software team that works while you sleep

You describe what you want built. A manager plans it, hires a team, and the team writes the code — on your accounts, on your servers, in your version control. This explains exactly how, with no coding knowledge assumed.

What it actually is

START HERE

Imagine hiring a small software agency. You give them a description of what you want. A manager reads it, works out the pieces, decides who is needed, and hands each person a task. They go away, write the code, test it, and submit it for review. The manager checks the work, sends back what is not good enough, and approves what is. Every so often they come back and ask you what you think.

devteam is that agency, staffed by AI instead of people. The manager is an AI. The engineers are AI. They are not a single chatbot pretending to be a team — they are genuinely separate programs, running at the same time, each with its own instructions, its own copy of the code, and no ability to see inside each other's heads. They coordinate the way a real team does: through tasks, written handover notes, reviews and pull requests.

The thing that makes it unusual is not the AI. It is the ownership. Most AI coding products run on the vendor's computers, with the vendor's accounts, and your code passes through their systems. devteam runs on *your* computer or *your* cloud account, spends *your* AI credits, and pushes to *your* code repository. Nothing is hosted for you, and there is no account to sign up for.

THE SHORT VERSION

A manager AI plans the work and supervises. Worker AIs do the building. A plain, non-AI program does the scheduling and the bookkeeping, because that part should be predictable and should not cost money every time it runs.

Why "your own" matters

THE FOUNDING IDEA

Three things belong to you, and each one is a deliberate choice with a real consequence.

Your own AI keys

An **API key** is a password that lets a program use an AI model and bills you for it. In devteam you supply your own. That means you see the exact cost, you can switch models, and no one is reselling you access at a markup. It also means each person using the system spends their own money — the software deliberately refuses to let one user's projects run on another user's credit.

It supports Claude, OpenAI and Gemini, and can also talk to an AI model you run yourself on your own hardware.

Your own cloud

The whole system is a program you run. On a laptop for trying it out; on a rented server for running it properly. There is no version of this where your code sits on somebody else's machine because that is the only way it works.

Your own version control

Version control is the system that stores code and remembers every change ever made to it — the industry's shared filing cabinet. GitHub is the best-known one. devteam works with GitHub, with a company's private GitHub, and with self-hosted alternatives. If your organisation keeps its code inside its own network, the team can work there without anything leaving.

WHAT THIS MEANS FOR YOU

You are the client, and you are also the landlord. Nothing runs that you did not start, nothing is billed that you did not authorise, and if you switch the server off, the entire system is off. There is no service still running somewhere with your code in it.

Who is on the team

THE CAST

Five kinds of participant. Understanding who does what is most of understanding the system.

You THE BOSS You write the brief, answer questions when asked, review what comes back, and decide when something is finished. How often you are interrupted is a setting you choose.	The manager AI · PLANS AND SUPERVISES Reads your brief, breaks it into tasks, works out what depends on what, assigns the work, reviews it, and decides whether it ships. Deliberately given no ability to write code — it can only manage.
The teammates AI · DO THE BUILDING Named people with a job, a personality, and a memory of what they have already built on this project. Each works alone in their own private copy of the code.	The scheduler ORDINARY SOFTWARE · NO AI The office administrator. Decides what can start now, launches it, notices when something has gone quiet, and keeps the records. Costs nothing to run because no AI is involved.
The round table AI · OPTIONAL, BEFORE YOU START A circle of AIs from different companies who argue about your idea before anyone builds anything, and produce a plan. Optional, and useful when you are not yet sure what you want.	

Why the manager cannot write code

This is one of the most important design decisions in the system, and it is easy to miss. The manager physically cannot open a file or run a command. It can only create tasks, read reports, ask you questions, send work back, and approve things.

The reason is that an AI which can both assign work and do the work will, under pressure, quietly do the work itself and report that the team delivered it. That is not hypothetical — it happened during development, and the manager reported completed work that no one had built. Removing the ability removes the temptation.

Why teammates have names

Each teammate is a real, persistent record: a name, a role, a personality, which AI model they run on, and short notes about what they have built so far. When a task comes back for a second attempt, it goes to the same person, who still remembers the first attempt.

This sounds cosmetic and is not. "Send it back to Priya, she wrote it" is a different instruction from "run the backend task again". The first keeps the context; the second throws it away and pays to rediscover it.

The life of a project

START TO FINISH, IN ORDER

This is the sequence every project follows. The numbers matter here — each step depends on the one before it.

01 You write a brief

YOU

A paragraph or two describing what you want. You also choose how many rounds of work to run, how independent the team should be, and — optionally — who is on the team and what they are like.

02 **Optionally, a round table argues about it first**

SEVERAL AIS, DIFFERENT COMPANIES

If your idea is still fuzzy, you can convene a round table. Between three and eight AIs from different companies each propose an approach, then critique each other's, then revise. Six is the recommended ceiling — past that you are mostly paying for agreement. A moderator writes up what survived, what was rejected and why, the risks, and what kind of team the work needs.

Using AIs from genuinely different companies is the point: two copies of the same model tend to agree with each other, and agreement is not the same as being right.

03 **The manager plans the work**

MANAGER

It breaks the brief into concrete tasks and records which ones must wait for others. "Build the login screen" waits for "build the login system"; two unrelated features do not wait for each other and run side by side.

This produces a map of the work — what can start immediately, and what is blocked behind something else.

04 **The team is hired**

MANAGER AND SCHEDULER

Roles become actual named people with personalities. If the round table decided this project needed someone who distrusts data they did not check themselves, that instruction is written into that teammate's standing brief and shapes everything they do.

05 **Tasks are handed out**

SCHEDULER

Anything not waiting on something else is started, up to a limit you set on how many can run at once. When it cannot start everything, it starts whatever is holding up the most other work — the task three people are waiting behind goes before the one nobody needs.

06

A teammate does the work

TEAMMATE

They get their own private copy of the code, read the handover notes from whoever they are building on top of, and work: reading files, writing files, running commands, testing as they go. If they get stuck, they can ask a specific colleague — who answers with their own actual work in front of them, not as a generic helper.

When finished, they save their work on a separate `branch` — a parallel version of the code that cannot affect anything until it is approved.

07

The platform tests it — not the AI

ORDINARY SOFTWARE · NO AI

This is the single most important safeguard. The project's own tests are run by the plain, non-AI part of the system, and the raw result is recorded before the AI writes its summary.

Without this, you have only the AI's word that it works, and AIs are famously willing to describe broken code in glowing terms. With it, the manager is judging evidence rather than a claim. When tests fail, the exact failures are named, so the feedback is "this specific check broke" rather than "the tests failed".

08

The work is reviewed

MANAGER, AND OPTIONALLY TEAMMATES

The manager reads the test result first and the write-up second. It can also ask teammates to review — they read independently, never seeing each other's opinions, and the author is never asked to review their own work.

The manager is shown the disagreement, not a verdict: "two of three would approve, and here is why the third would not". The disagreement is the part it could not have worked out alone.

09

Approved, or sent back

MANAGER

Approved work becomes a **pull request** — a formal proposal to add the change to the main codebase, which you can read and comment on.

Rejected work goes back to the same teammate with specific feedback.

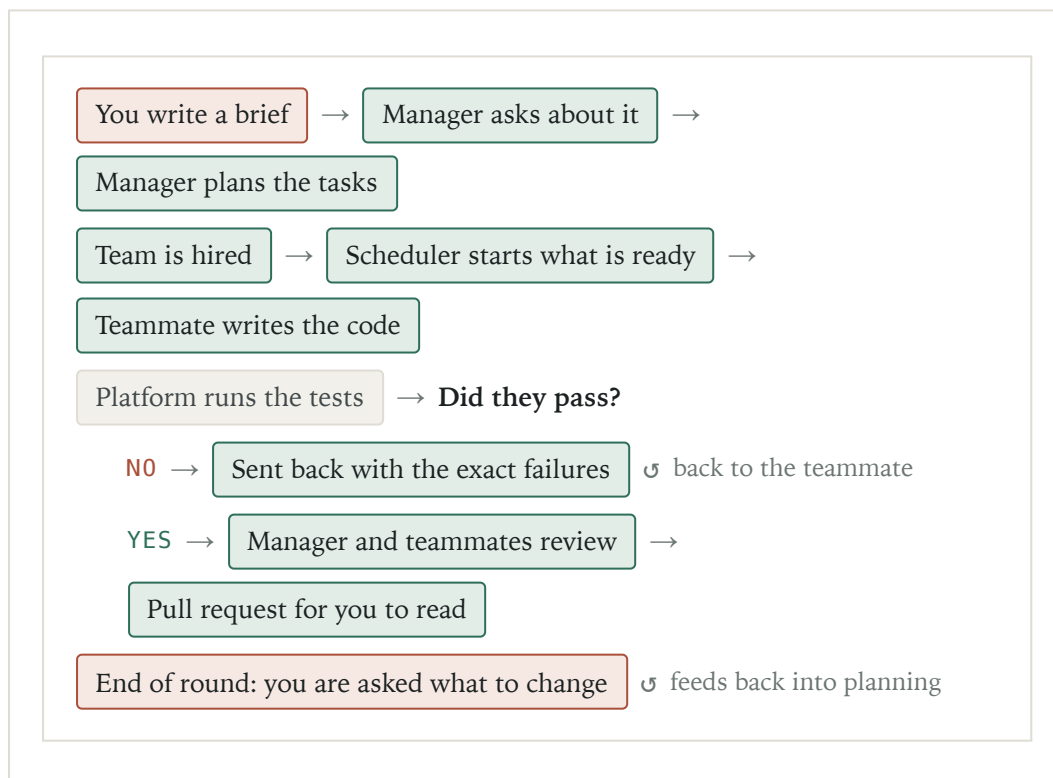
If something fails twice, it moves to a more capable and more expensive AI model. If it keeps failing, that is treated as a planning problem rather than an effort problem, and the manager is told to rethink rather than retry.

10

The round ends and you are asked

YOU, IF YOU ARE AROUND

At the end of each round of work, you get a summary of what shipped and are asked whether anything should change before the next round. If you do not answer, it carries on — an overnight run must not stall waiting for someone who is asleep. Your answer is read whenever it arrives.

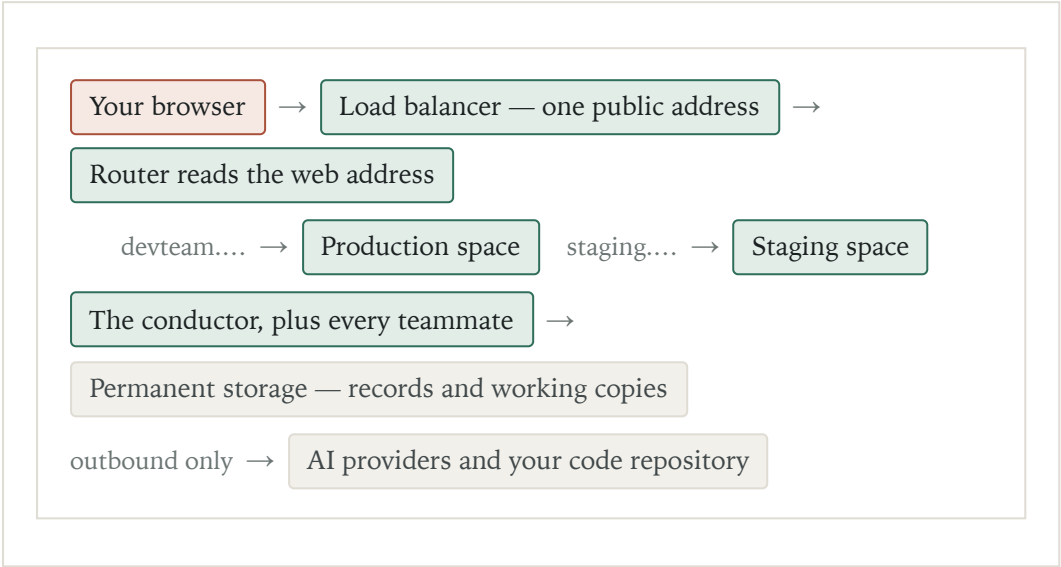


The loop a project runs in. The only two places a human appears are writing the brief and answering at the end of a round – everything else runs without you.

What the machine looks like

ARCHITECTURE, IN PLAIN TERMS

Six pieces. Only two of them cost money to run.



Everything inside "your server" is one program you run. The teammates are separate programs it starts and stops. The only things reaching outside are AI requests and code being saved.

PIECE	WHAT IT DOES	COSTS MONEY?
Dashboard	The web page you look at. Shows projects, live activity, the team, costs and results.	No
Conductor	The central program. Serves the dashboard, holds the records, starts everything else.	No
Records	A single file on disk holding every project, task, teammate, message and result.	No
Scheduler	Decides what starts when, notices stalled work, opens pull requests automatically.	No
Manager	The AI that plans and supervises.	Yes
Teammates	Separate AI programs that write the code, one per task.	Yes

Why the scheduler is not an AI

Deciding "task 4 is waiting on tasks 1 and 2, both are finished, so task 4 can start" is arithmetic. Asking an AI to do it would cost money on every check, take seconds instead of milliseconds, and occasionally get it wrong in a way that is hard to spot.

So the rule is: **AI is used for judgement, ordinary software for bookkeeping.**

Which tasks can start, who is free, what has gone quiet, what has already been paid for — all bookkeeping. Is this code good enough, what should we build next, is this failure the model's fault or the plan's — all judgement.

Why teammates are separate programs

Each teammate runs as its own program with its own private copy of the code. It cannot see or disturb what anyone else is doing. If one crashes, hangs, or runs up an unexpected bill, it is killed on its own without touching the rest.

This also makes a genuine failure mode visible rather than mysterious: if two teammates were ever pointed at the same file at the same time, they would overwrite each other. The system treats that as a dependency — one waits for the other — rather than pretending it is parallel work.

How you see what is happening

The dashboard keeps a live connection open to the conductor, so activity appears as it happens rather than when you refresh. Every meaningful event is also written down permanently, so you can read back through exactly what occurred and when, long after the fact.

Under the bonnet

FOR A TECHNICAL READER, OR A CURIOUS ONE

Everything above is true without knowing any of this. This section is for someone who needs to evaluate, host or extend the thing — or who simply wants to know what the words mean.

What "the conductor" actually is

The conductor is one program: a Python web server of about 13,500 lines that does five jobs and no others.

- **Serves the dashboard.** The pages you look at are plain HTML, CSS and JavaScript files read straight off the disk. There is no build step and no framework — you can edit a file and reload the page.

- **Answers the API.** Every action in the dashboard is a request to the conductor.
- **Holds the records.** A single `SQLite` file — a whole database in one file, needing no separate database server to be installed or maintained.
- **Runs the manager.** The manager's AI session lives inside the conductor's own process.
- **Starts and stops teammates.** Each is launched as a separate program and watched.

It is deliberately one process rather than a collection of services. A system where the scheduler, the API and the record-keeping are three separate programs that must find each other over a network has three ways to be half-running. This has one: it is up, or it is not.

The API, in families

An `API` is the list of requests a program will answer — the difference between a system you can only click and one you can drive. There are 108. You never need to touch them; the dashboard uses them for you. They group like this:

FAMILY	EXAMPLE	WHAT IT IS FOR
Projects	<code>/api/projects</code>	Create, list, cancel, restart. Set the budget and how independent the team is.
Tasks & activity	<code>/api/projects/1/events</code>	The live feed, task detail, and the raw logs from a specific agent.
The team	<code>/api/projects/1/team</code>	Who is on it, what they have built, and changing someone's personality or model.
Talking to the manager	<code>/api/projects/1/directive</code>	Send an instruction, answer a question it asked, leave notes on a specific piece of work.
Output	<code>/api/projects/1/artifacts</code>	What was produced, per sprint and per teammate; files; release notes.
Measurement	<code>/api/projects/1/metrics</code>	First-attempt acceptance, rework, cost per delivered task.
Settings	<code>/api/tuning</code>	The dials that govern how the team behaves, and your keys and endpoints.
Deployment	<code>/api/projects/1/deploy</code>	Run a built app, preview a branch, roll back.
Self-repair	<code>/api/self/...</code>	The platform working on itself: findings, staging, update, rollback. Restricted to the owner.

FAMILY	EXAMPLE	WHAT IT IS FOR
Health	/api/health	Answers only if the database also answers. Watched from outside.

Three doors, three different keys

Not everything is protected the same way, and the differences are deliberate.

DOOR	WHO GETS IN	WHY IT IS SEPARATE
/api/...	You, after signing in	Normal use. Limited login attempts, and a lockout that survives a restart.
/internal/...	Teammates only, with a shared token	Just three requests: report finished work, post activity, look up a colleague. A teammate is not a user and should not hold a user's powers.
/api/health	Anyone	A monitor that needed to sign in could not tell you that signing in was broken.

The live feed

The dashboard holds one long-lived connection open — a `WebSocket`, which unlike a normal request stays open so the server can speak first. That is why activity appears as it happens rather than when you refresh. Every event is also written to the database, so nothing depends on you having been watching.

What the database holds

Eighteen tables in one file. The ones worth knowing: *projects* and *tasks* (the work), *agents* (the teammates and their memory), *runs* (one row per AI task, which is where all the measurement comes from), *events* (the permanent activity record), *findings* (what the platform believes is wrong with itself), *sprint_artifacts* (the sealed record of each round), *feedback* (your notes, attached to specific work), and *tuning* (the dials, so they can be changed without rebuilding anything).

Where things actually run

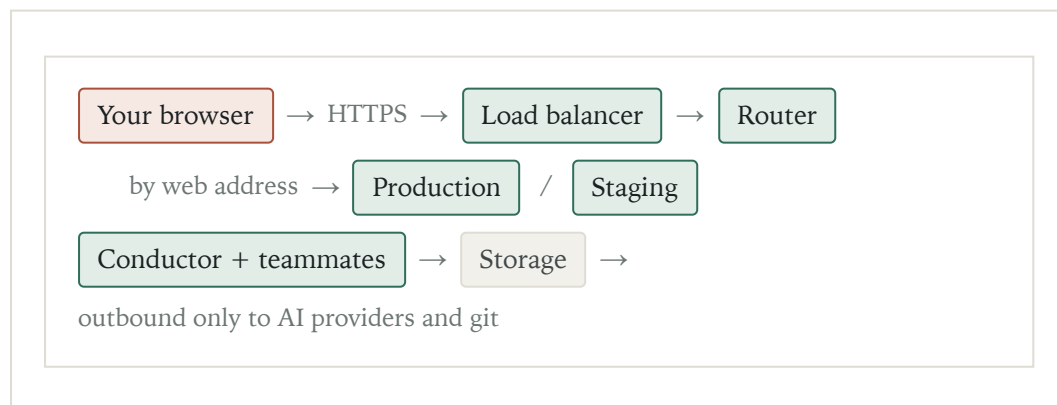
The same code runs in four shapes, and they are genuinely different.

SHAPE	WHERE	WHAT IS ISOLATED
Local	A program on your laptop	Nothing. Your real records, your real keys.
Sandbox	A second program on your laptop	Its own records, and every credential deliberately blanked. Used to click through a proposed change to the platform itself before accepting it.
Staging	Its own space on the cluster	Own records and web address. Real keys, but not permitted to approve its own changes. Its storage is deliberately throwaway — a restart wipes it, which is what you want of a rehearsal.
Production	Its own space on the cluster	Own storage, holds the real credentials. The live system.

The sandbox is worth a sentence of its own, because the obvious guess is wrong: it is not a miniature cluster. It is one extra copy of the program on the same machine, with its records replaced by fake ones and every credential blanked out — *blanked, not removed*, because a program inherits its parent's settings, and an unset key would silently fall through to the real one.

The network path, end to end

Following a single click from your browser to the code:



One public address serves both environments. Which one you reach is decided by the web address you typed, not by a separate machine — which is why staging costs almost nothing to keep running.

- **One public entrance.** A single load balancer holds the public address. Not the largest line on the bill, but the one you pay for *per environment* — which is exactly why there is one and everything shares it.
- **A router decides where to go.** It reads the web address you asked for and forwards to the right environment. Adding a new environment costs nothing extra.
- **Encryption is automatic.** Certificates are requested and renewed without anyone doing anything, and you are emailed if one is ever close to expiring.
- **Traffic only goes outward.** Nothing on the internet can start a conversation with a teammate. Teammates reach out to AI providers and to your code repository; nothing reaches in.
- **Storage outlives the program.** Records and working copies sit on a separate disk that survives restarts and updates. Updating the software does not touch your data.

WORTH KNOWING BEFORE YOU HOST IT

The public entrance means the sign-in page is reachable from the internet, which makes your password the entire security boundary. Login attempts are limited and the limit survives restarts — but the password itself is the thing to take seriously, and the system will tell you when you have chosen a weak one.

How an update actually reaches the server

Code is packaged into a **container image** — a frozen, complete copy of the program and everything it needs, identified by a fingerprint of its own contents. That fingerprint matters: two builds of identical code produce the same identity, so "is this the version I tested?" has a real answer rather than a guess based on timestamps.

The image is uploaded to a private registry, tried on staging, run once as a canary with no traffic, and only then does the live system switch to it. The switch changes one thing — which version runs — rather than re-applying the whole configuration, because re-applying has been observed to quietly roll the system backwards to whatever version the configuration file happened to name.

How the manager decides

THE JUDGEMENT CALLS

This part is deliberately grounded in published research on how multiple AIs working together actually perform, rather than in intuition about what sounds

clever.

More opinions, gathered properly

The tempting idea is: run the same task three times with three different AIs and pick the best result. That does help, but far less than expected. The research finding that shaped this system is that **how you combine the opinions matters several times more than how many opinions you gather**.

So the system spends its effort on the combining. One teammate reviews by default, and that number is a setting rather than a constant. Reviewers read the work independently and never see each other's opinions — because a second reviewer who reads the first one's verdict tends to agree with it, and you have paid twice for one opinion. Their disagreements are preserved and handed to the manager rather than averaged away.

Competitions, used sparingly

For a genuinely uncertain task, the manager can run two or three teammates on it at once, each on a different AI model and each in their own copy of the code. It then compares the results and picks a winner. This is capped deliberately low, because the value comes from judging well, not from the number of entries.

Cheap first, expensive when earned

Work starts on a fast, cheap model. If it fails twice, it moves up to a stronger one, and then to the strongest. The ladder is relative to whatever just failed, so "escalating" never accidentally means running the same model again.

Crucially, the system distinguishes between *the AI could not do this* and *the AI provider was too busy right now*. The second is not a failure of quality, and treating it as one burns money moving to an expensive model to solve a problem that would have cleared up on its own. Capacity problems get a retry; quality problems get an upgrade.

Working in slices, or in layers

You choose how the work is broken up, and both options have a real cost the manager is told about.

Agile the default	By outcome. Every task delivers something you can actually see working, however narrow. Something runs on day one.	More friction, because slices keep touching the same files.
--------------------------------	--	---

Waterfall	By layer. Foundations first, then what sits on them. Right when the shape is already known — a migration, or building to a written specification.	Nothing is demonstrable until late, so a misunderstanding of your brief survives until it is expensive.
------------------	---	---

Everything is measured

Every single time an AI is asked to do a task, the system records what it was, which model, which attempt number, what it cost, how long it took, and whether the work was ultimately accepted.

From that it can answer the question that actually matters: **how often is work accepted on the first attempt?** That number is the honest measure of whether the planning and briefing are working, because it is the one thing that cannot be improved by simply spending more money. Two settings can then be compared directly rather than argued about.

WHY THIS IS UNUSUAL

Most AI tooling reports what it cost. This reports how much of what it cost was wasted — work that was done, paid for, and then thrown away and redone. That is the number you tune against.

Sprints and your involvement

HOW PRESENT YOU NEED TO BE

Work runs in **sprints** — rounds of building, testing and shipping. You say roughly how many you want at the start; the manager can revise that number once it understands the work, because your original guess was made before anyone had read anything.

Three levels of independence

MODE	WHAT HAPPENS	SUITS
Ask me	Stops and waits for you at every meaningful decision.	Watching it work; anything you care about deeply.
Supervised	Proceeds on its own, but stops for decisions that could paper over a problem.	Most work.
Autonomous	Decides everything itself, but records every judgement call so you can audit it afterwards.	Overnight runs.

The decisions it always flags

Some decisions are dangerous specifically because a confident manager makes them smoothly. Three are singled out: approving work nobody actually delivered, shipping code whose tests are failing, and giving up on a task after repeated failure. In supervised mode these stop and ask you. In autonomous mode they proceed — you asked for that — but each is recorded loudly and permanently for review.

BEING ASKED WITHOUT BEING BLOCKED

At the end of each sprint you are asked what should change before the next one is planned. This is the cheapest possible moment to redirect: nothing has been built yet, so changing course costs nothing, whereas the same correction three sprints later means discarding finished work.

The question does not block. An unattended overnight run that stalled an hour per sprint waiting for you would have destroyed the thing you asked for. Your answer is read whenever it arrives, even days later — and it is labelled with which sprint it was about, so nobody tries to rewrite history that has already been sealed.

Where the work ends up

WHAT YOU ACTUALLY GET

Code, in your repository

Every task's work lives on its own branch and arrives as a pull request — a formal, reviewable proposal you can read line by line, comment on, and accept or reject. Nothing lands in your main codebase without passing through that gate.

A record of each sprint, frozen

When a sprint ends, what it produced is sealed: which tasks were delivered, by whom, with which pull request, and whether the tests passed. Later sprints cannot rewrite it. Without this, sprint three quietly overwrites sprint one and you lose the ability to say what shipped when.

Release notes built from facts

Each sprint gets a summary of what changed, assembled from what actually happened — the delivered tasks, their pull requests, their test results. Every line traces back to a real task. If an AI is used to write the prose, its draft is discarded entirely if it mentions work the sprint did not contain, and the underlying facts stay visible next to it. A release note that cannot be traced back to a task is worse than no release note.

Documents, for non-code projects

The system is not limited to software — it can produce research, strategy documents and plans. These are rendered properly rather than shown as raw text.

Running previews

A branch can be deployed on its own so you can click through a change before deciding on it, without disturbing the live version.

How it looks after itself

THE SELF-REPAIR LOOP

devteam's own source code can be given to devteam as a project. It can read its own code, plan a fix, write it, test it, and propose it — the same as any other project, through the same review gate.

The daily check

Once a day, without being asked, it reviews everything that went wrong: failures to start work, agents that went silent, errors in the dashboard, requests that failed. Each distinct fault becomes a tracked item that is counted rather than merely logged.

This distinction matters more than it sounds. Previously all of this was recorded in a running activity feed — visible only to someone already watching it. A fault that happened forty times over a weekend looked exactly like one that happened once, because neither was ever counted.

How it ranks what is wrong

- **Severity sets the band, frequency ranks within it.** A rare serious fault always outranks a common trivial one. Ranking purely by frequency would put a chatty warning above a data-loss bug.
- **Old problems sink.** Something that stopped happening a week ago was probably fixed by something else. Without this the top of the list becomes a museum.
- **A problem that comes back after being fixed reopens loudly.** A repair that did not hold is more interesting than a fresh fault, and quietly re-filing it as new would hide that.

What it will and will not do alone

Looking, counting and ranking are free and happen automatically. Raising a ticket is cheap and reversible and happens automatically. **Rewriting its own running code is neither, and does not happen because something scored highly.** An unattended process that both decides what is wrong and rewrites the program it is running inside, with nobody watching, is a specific and bad idea unless the operator has explicitly said otherwise.

Updating itself safely

When a new version is ready, there is a sequence and each step can stop it:

STAGE	WHAT HAPPENS
Staging	A complete second copy, with its own separate database, running the new version. It can propose changes but is not permitted to approve them.
The full test suite	Over six hundred automated checks run inside the new version itself, not on a developer's laptop.
Canary	The new version is started once, privately, with no traffic sent to it, purely to see whether it starts and answers at all. It is then removed either way.
Promotion	Only now does the live version change, and only by swapping which version runs — never by re-applying the whole configuration, which has been observed to silently roll a system backwards.
Rollback	One step back to the previously known-good version, kept in history rather than rebuilt. Rebuilding to go backwards produces a <i>new</i> version from old instructions, not the one you know worked.

Watched from outside

A separate service checks the live site from three continents every few minutes and emails if it stops answering, or if its security certificate is close to expiring. This has to come from outside for a reason that is obvious once said: **the platform cannot send you a message telling you it has crashed.**

The safety rails

WHAT STOPS IT RUNNING AWAY

RAIL

WHAT IT PREVENTS

A limit on AI tasks per project	A stuck loop burning through your credits overnight. This is the primary limit — on a flat-rate subscription it is more meaningful than a money cap.
A spending cap	The same thing, in money, when you are billed per use.
Tests run by the platform	An AI describing broken code as working. The exit code is evidence; the write-up is a claim.
The manager cannot write code	The manager quietly doing the work itself and reporting that the team delivered it.
Staging cannot approve its own changes	A test environment accidentally shipping to production.
Everyone spends their own credits	One user's work being billed to another's account.
Limited login attempts	Someone guessing your password. This survives restarts — it used to reset every time the system restarted, which meant an attacker only had to wait for an ordinary update.
A watchdog on silent work	A crashed teammate leaving a project waiting forever for a report that will never arrive.
Disk space limits	Accumulated copies of code filling the disk, which stops everything, because every teammate needs somewhere to work.

One honest caveat about permissions

Teammates run with broad permissions inside their own copy of the code — they need to run commands to build and test things, and asking permission for each would defeat the purpose.

It is worth being precise about how much separation that copy actually gives, because the intuitive answer is wrong. **As currently deployed, a teammate is a separate process, not a separate sealed box.** Every teammate runs inside the same container as the conductor, sharing its filesystem and its memory allowance. They cannot overwrite each other's work, because each gets its own directory — but a teammate that went badly wrong could read the database file, or another teammate's working directory.

The stronger arrangement exists and is built: each teammate gets its own sealed box, scheduled separately. It is a configuration change, not new work. It is not what runs today, because one shared box is simpler and has been sufficient — and that trade should be a decision you make knowingly rather than one you inherit from a document that implied otherwise.

READ THIS AS

The boundary that is real today is around the *whole system*: the container it runs in, and the server you put it on. The boundary between one teammate and another is about tidiness and crash isolation, not security.

What it costs

TWO SEPARATE BILLS

There is no charge for the software. You pay two other people.

The server

About **\$42 a month**, or \$1.40 a day. The breakdown is worth seeing, because the shape of it is the interesting part.

ITEM	MONTHLY
One machine — 2 processors, 4 GB memory	\$24
Load balancer — the public entrance	\$12
Private registry, holding the packaged versions	\$5
Storage, 11 GB	~\$1

The machine is the biggest single line, but the **load balancer is the one that multiplies**. You pay per entrance, so giving staging its own would add another \$12

for something serving a handful of requests a day. That is why every environment shares one — and why adding a fourth costs nothing at all.

The AI

Billed by your AI provider, directly to you. In a measured real run, a complete small feature — written, tested, and submitted for review — cost around **ten cents**. A retry after a provider outage took it to roughly forty cents. Larger work costs more, and the per-project limits are there so it cannot surprise you.

On a flat-rate AI subscription there is no per-task charge at all, and the meaningful limit becomes how many tasks you are allowed, not what they cost.

Solid, and not yet

AN HONEST STATUS

Any documentation that only lists strengths is marketing. This is the real state of things.

Proven

- The full loop works end to end: brief, plan, build, test, review, pull request. It has been run live, repeatedly, with real accounts.
- It has shipped a fix to *itself* — written the change, tested it, and deployed it over its own running version.
- The safety check has been proven by deliberately giving it a broken version, which it correctly rejected while the live system carried on serving.
- Over seven hundred automated checks, all passing, including inside the deployed system rather than only on a developer's machine.
- It handles real-world trouble correctly: during testing an AI provider had an outage mid-task, and the manager correctly identified it as a temporary capacity problem rather than bad work, retried, and shipped.

Early

NOT FINISHED, STATED PLAINLY

- **Non-Claude teammates are unproven.** The ability to have OpenAI or Gemini write code is built and tested against simulated responses, but has never made a real call. Treat it as beta. Planning and discussion on those providers are proven; building is not.
- **Amazon and Google's enterprise AI services are configurable, not supported.** Both need a kind of authentication that is not simply a password pasted into a settings box. Saying otherwise would be a lie of omission.
- **Only one cloud provider is implemented.** The seam for adding others exists but nothing has been built through it, so it is untested by use.
- **The manager does not interview you before it starts.** It plans from your brief alone. It asks between sprints, but not before the first one.

A note on how the bugs above were found

Three real defects were caught not by the seven hundred automated checks but by running the system for real on a staging copy. In one, a teammate's assigned AI model silently overrode the manager's decision to escalate a failing task — so the manager correctly diagnosed a problem, correctly issued a correction, and the system discarded it and repeated the same failure.

That is worth knowing as a user of any AI system: automated checks confirm the things you thought to check. Running it for real is what finds the rest.

Glossary

EVERY TECHNICAL TERM USED ABOVE

API key	A password that lets a program use a paid service, such as an AI model, and bills the owner of the key.
Branch	A parallel version of the code where someone can work without affecting anyone else. Merged in once approved, or thrown away.

Canary	Starting a new version once, privately, with no real traffic, purely to see whether it works before anyone relies on it.
API	The list of requests a program will answer — the difference between a system you can only click and one you can drive.
Cluster	A group of rented computers managed as one, so a program can be moved, restarted or scaled without anyone touching a specific machine.
Container	A sealed box containing a program and everything it needs. It behaves identically wherever it runs, and cannot easily reach outside itself.
Container image	A frozen, complete copy of a program and everything it needs, identified by a fingerprint of its own contents.
Load balancer	The single public entrance that accepts traffic from the internet and passes it inward. Usually the priciest line on a cloud bill.
Pull request	A formal proposal to add a change to the main codebase, which others can read, comment on, and approve or reject.
Repository	The stored home of a codebase, including the full history of every change ever made to it.
Rollback	Returning to the previous working version after a bad update.

SQLite	A complete database kept in a single file, needing no separate database server to install or maintain.
Sprint	One round of planning, building, testing and shipping. Borrowed from how human software teams organise their time.
Staging	A complete second copy of the system used to try changes safely before they reach the version people actually use.
Test suite	A collection of automated checks that confirm the software still does what it is supposed to. Run by the platform, not by the AI.
Version control	The system that stores code and remembers every change, who made it and why. GitHub is the best-known example.
WebSocket	A connection that stays open, so the server can send you something without being asked. How the live activity feed works.

This handbook describes devteam as built and running. Figures for cost and test counts are from measured runs rather than estimates. Where something is unfinished it is marked as such in "Solid, and not yet" – that section is not a disclaimer, it is part of the specification.